

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a36d140cdf794a3e5.jsonl`  
- SHA-256: `a24e27b5ed3a2ed9950c533b84ffcd6d3936badfc9bcdceb64f98c829fcd57a2`  
- Source modified: `2026-06-10T16:31:36+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:28:24.044Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: HDFC parser persists all rows even when the parse is UNRECONCILED, and `txns list` never filters by document status
SEVERITY CLAIMED: high
FILE: src/muneem/parsers/hdfc.py (line 435-458)
DESCRIPTION: Unlike AxisParser/AUParser (which store nothing on an unverified parse), HDFCParser.parse_and_insert inserts every transaction into hdfc_transactions BEFORE computing the verdict and never removes them when the verdict is 'unreconciled' ? the status is recorded but the suspect rows stay. The only read path, `muneem txns list hdfc` (cli.py:347-361 via db.list_hdfc_transactions, db.py:437-440), does not join documents or filter/annotate by status, so rows from a known-bad parse (e.g. a page whose 'Narration'/'Closing' header line failed to extract ? _table_body returns [] for that page and silently drops the whole page's transactions) are served indistinguishably from trusted rows. Related: _parse_transaction line 219 fabricates `closing\_balance=0.0` when the balance token fails _AMOUNT_RE, persisting an invented zero balance on such rows.
EVIDENCE QUOTED: hdfc.py:435-447: `for t in stmt.transactions:
dbmod.insert_hdfc_transaction(...)` runs unconditionally; :454 `verdict = Confidence.RECONCILED
if ok else Confidence.UNRECONCILED` only sets status. db.py:437-440: `SELECT * FROM
hdfc_transactions ORDER BY id DESC LIMIT ?` ? no status filter. hdfc.py:219:
`closing\_balance=closing if closing is not None else 0.0`.
SUGGESTED FIX: Apply the same gate as Axis/AU: compute the verdict first (parse + summary are both pure reads) and insert rows only when reconciled ? or at minimum exclude/flag rows whose document status != 'reconciled' in list queries. Replace the fabricated 0.0 closing balance with None or a parse rejection.

Your job is to try to REFUTE it. Your lens: existing-mitigation: check whether something ELSE in the repo (a test, a guard, a caller-side check, a hook, config) already prevents or detects the claimed problem.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Structured output only.

2. user (2026-06-10T16:28:30.721Z)

```
1      """HDFC savings / transaction-account statement parser.
2
3      **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
4      default ``extract_tables()`` and not a ready-made library. HDFC's transaction
5      table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
6      decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
7      position. pdfplumber's whole chain is permissive and it opens password-protected
8      statements directly.
9
10     **Layout.** Each transaction is one *primary* line ? the transaction date at the
11     far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
12     Closing Balance) ? followed by zero or more *continuation* lines that wrap the
13     narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
14     cheque/reference number. We group words into visual lines by ``top``, detect
15     primary lines by a leading date + a closing-balance amount, attach continuations,
16     and read columns by x-position.
17
18     **Correctness is checked, not assumed.** ``reconcile`` verifies the running
19     balance (closing[i] == closing[i-1] - withdrawal + deposit) and
20     ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
21     statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
22     parse complete without inspecting any individual value ? which is how we validate
23     against real (sensitive) statements. No statement content is logged here.
24     """
25
26     from __future__ import annotations
27
28     import csv
29     import re
30     from dataclasses import dataclass, field
31     from pathlib import Path
32
33     import pdfplumber
34
35     from muneem import __version__
36     from muneem import db as dbmod
37     from muneem.errors import ParseError
38     from muneem.redaction import redact_filename
39
40     from . import register_parser
41     from .base import StatementParser
42     from .concepts import ConceptRow, StatementConcepts
43     from .profiles import HDFC_PROFILE
44     from .validation import (
45         BookendReport,
46         Confidence,
47         ReconcileReport,
48         reconcile_running_balance,
49         verify,
50     )
51
52     # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
53     # Left-aligned columns key on x0; right-aligned amount columns key on their right
54     # edge (x1), which is stable regardless of the number's magnitude.
55     _DATE_MAX_X0 = 70.0 # transaction date sits at x0?52; the value date is at x0>110
56     _NARR_MIN_X0 = 100.0 # narration band starts at x0?111
57     _AMOUNT_MIN_X0 = 295.0 # amount columns start at x0?306; narration never crosses ?290
58     _WITHDRAWAL_MAX_X1 = 400.0 # withdrawal right edge ?358
59     _DEPOSIT_MAX_X1 = 490.0 # deposit right edge ?443; closing balance right edge ?561
60     _LINE_TOL = 2.5 # words within this many points of each other share a visual line
61     _CONT_MAX_SPAN = 70.0 # a continuation line is at most this far below its primary line
```

```

62
63     _DATE_RE = re.compile(r"^d{2}/d{2}/d{4}$")
64     # Indian-format money: optional minus, comma groups, exactly two decimals, optional
Cr/Dr.
65     _AMOUNT_RE = re.compile(r"^-[d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
66     _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}
67
68
69     @dataclass
70     class Transaction:
71         date: str # dd/mm/yyyy, as printed
72         narration: str
73         withdrawal: float
74         deposit: float
75         closing_balance: float
76         value_date: str | None = None
77         ref: str | None = None
78         page: int = 0
79
80         def to_concept(self) -> ConceptRow:
81             """Map HDFC's native row onto the bank-agnostic concept vocabulary (the
verifier's input).
82
83             Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
``page``
84             field is HDFC-internal provenance and has no concept equivalent.
85             """
86             return ConceptRow(
87                 txn_date=self.date,
88                 narration=self.narration,
89                 debit=self.withdrawal,
90                 credit=self.deposit,
91                 balance=self.closing_balance,
92                 value_date=self.value_date,
93                 reference=self.ref,
94             )
95
96
97     @dataclass
98     class Statement:
99         path: Path
100         transactions: list[Transaction] = field(default_factory=list)
101         account_number: str | None = None
102
103
104     # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept
vocabulary +
105     # verifier (the concept-core refactor). These back-compat names keep HDFC's public
surface stable:
106     # StatementSummary == the statement-level concepts a SUMMARY block supplies
107     # ValidationReport == the verifier's bookend report (per-row walk + printed-figure
checks)
108     StatementSummary = StatementConcepts
109     ValidationReport = BookendReport
110
111
112     def _to_amount(token: str) -> float | None:
113         """Parse an HDFC money token to a float, or None if it is not money.
114
115         Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
116         is real information and the reconciliation math needs it.
117         """
118         if not _AMOUNT_RE.match(token):
119             return None
120         cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "")

```

```

121     try:
122         return float(cleaned)
123     except ValueError:
124         return None
125
126
127 def _group_lines(words: list[dict]) -> list[list[dict]]:
128     """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
129     lines: list[list[dict]] = []
130     anchor_top: float | None = None
131     for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
132         if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
133             lines[-1].append(w)
134         else:
135             lines.append([w])
136             anchor_top = w["top"]
137     return [sorted(line, key=lambda w: w["x0"]) for line in lines]
138
139
140 def _line_top(line: list[dict]) -> float:
141     return min(w["top"] for w in line)
142
143
144 def _has_narration(line: list[dict]) -> bool:
145     return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)
146
147
148 def _has_amount(line: list[dict]) -> bool:
149     return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)
150
151
152 def _has_leading_date(line: list[dict]) -> bool:
153     first = line[0]
154     return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)
155
156
157 def _has_footer_marker(line: list[dict]) -> bool:
158     return any(w["text"].upper() in _FOOTER_MARKERS for w in line)
159
160
161 def _is_primary(line: list[dict]) -> bool:
162     """A transaction's first line: a date at the far left + a closing-balance amount.
163
164     Requiring the closing-balance amount guards against stray dates in the page
165     header / account-summary box (e.g. the statement period).
166     """
167     if not _has_leading_date(line):
168         return False
169     return any(
170         w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
171         _AMOUNT_RE.match(w["text"])
172         for w in line
173     )
174
175 def _is_continuation(line: list[dict]) -> bool:
176     """A narration-wrap line: narration-band text, no leading date, no amounts."""
177     return _has_narration(line) and not _has_leading_date(line) and not
178     _has_amount(line)
179
180 def _parse_transaction(
181     primary: list[dict], continuations: list[list[dict]], page_no: int
182 ) -> Transaction:
183     withdrawal = deposit = 0.0

```

```

184     closing: float | None = None
185     narr: list[tuple[float, float, str]] = [] # (top, x0, text) for ordered reassembly
186
187     for line in (primary, *continuations):
188         for w in line:
189             x0, x1, text = w["x0"], w["x1"], w["text"]
190             if x0 >= _AMOUNT_MIN_X0:
191                 val = _to_amount(text)
192                 if val is None:
193                     continue
194                 if x1 < _WITHDRAWAL_MAX_X1:
195                     withdrawal = val
196                 elif x1 < _DEPOSIT_MAX_X1:
197                     deposit = val
198                 else:
199                     closing = val
200             elif x0 >= _NARR_MIN_X0:
201                 narr.append((w["top"], x0, text))
202
203     value_date: str | None = None
204     parts: list[str] = []
205     for _top, _x0, text in sorted(narr):
206         if value_date is None and _DATE_RE.match(text):
207             value_date = text # the value date, lifted out of the narration band
208             continue
209         parts.append(text)
210
211     ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
212     ref = max(ref_candidates, key=len) if ref_candidates else None
213
214     return Transaction(
215         date=primary[0]["text"],
216         narration=" ".join(parts),
217         withdrawal=withdrawal,
218         deposit=deposit,
219         closing_balance=closing if closing is not None else 0.0,
220         value_date=value_date,
221         ref=ref,
222         page=page_no,
223     )
224
225
226     def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
list[Transaction]:
227         """Pure, PDF-free core: turn grouped table-body lines into transactions.
228
229         Kept separate from PDF I/O so the column logic is unit-testable on synthetic
230         word lists, with no real data.
231         """
232         primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
233         transactions: list[Transaction] = []
234         for k, idx in enumerate(primaries):
235             stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
236             primary = lines[idx]
237             primary_top = _line_top(primary)
238             continuations: list[list[dict]] = []
239             for m in range(idx + 1, stop):
240                 line = lines[m]
241                 if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
242                     break
243                 if _is_continuation(line):
244                     continuations.append(line)
245                 transactions.append(_parse_transaction(primary, continuations, page_no))
246         return transactions

```

```

247
248
249 def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
250     """Lines below the column header (drops the per-page account-summary box)."""
251     header_top: float | None = None
252     for line in lines:
253         texts = {w["text"] for w in line}
254         if "Narration" in texts and any(t.startswith("Closing") for t in texts):
255             header_top = _line_top(line)
256             break
257     if header_top is None:
258         return []
259     return [line for line in lines if _line_top(line) > header_top + 1]
260
261
262 def _extract_account_number(text: str) -> str | None:
263     """Account number via the HDFC profile (bank knowledge as data; same pattern as
264 before)."""
265     return HDFC_PROFILE.first_account(text)
266
267
268 def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
269     """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
270
271     ``password`` is forwarded to pdfplumber for encrypted statements and used
272     in-memory only ? never stored or logged.
273     """
274     path = Path(pdf_path)
275     transactions: list[Transaction] = []
276     account_number: str | None = None
277     with pdfplumber.open(str(path), password=password or "") as pdf:
278         for page_no, page in enumerate(pdf.pages):
279             if account_number is None and page_no < 2:
280                 account_number = _extract_account_number(page.extract_text() or "")
281             words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
282             body = _table_body(_group_lines(words))
283             transactions.extend(extract_transactions_from_lines(body, page_no))
284     return Statement(path=path, transactions=transactions,
285 account_number=account_number)
286
287
288 def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
289     """Check the running balance via the shared reconciler (closing[i] ==
290 closing[i-1] - withdrawal + deposit).
291
292     Spans page breaks (transactions are in document order), so a clean report also
293     validates page stitching. Reports counts only ? no amounts.
294     """
295     entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
296 statement.transactions]
297     return reconcile_running_balance(entries, tol)
298
299
300 def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
301     """Pure core: read HDFC's SUMMARY block from extracted text lines.
302
303     Layout (stable across statements)::
304
305         SUMMARY
306         Opening Balance  Debit Amount  Credit Amount  Closing Balance
307         <opening>         <debitAmt>   <creditAmt>   <closing>
308         Debit Count      Credit Count
309         <debitCount> <creditCount>
310
311     The values land on their own lines, so we take the first post-SUMMARY line

```

```

309     carrying ?4 money tokens (the balances) and the first carrying ?2 bare
310     integers and no money (the counts).
311     """
312     start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
313     if start is None:
314         return None
315     balances: list[float] | None = None
316     counts: list[int] | None = None
317     for ln in lines[start + 1 : start + 8]:
318         tokens = ln.split()
319         monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
320         ints = [int(t) for t in tokens if t.isdigit()]
321         if balances is None and len(monies) >= 4:
322             balances = monies[:4]
323         elif counts is None and not monies and len(ints) >= 2:
324             counts = ints[:2]
325     if balances is None and counts is None:
326         return None
327     bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
328     opening, debit_amt, credit_amt, closing = bvals[:4]
329     cvals: list[int | None] = [*counts] if counts else [None, None]
330     debit_count, credit_count = cvals[:2]
331     return StatementSummary(
332         opening_balance=opening,
333         closing_balance=closing,
334         debit_count=debit_count,
335         credit_count=credit_count,
336         debit_amount=debit_amt,
337         credit_amount=credit_amt,
338     )
339
340
341 def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
342     """Read the bank's own SUMMARY figures from the statement, or None if absent."""
343     with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
344         for page in pdf.pages:
345             text = page.extract_text() or ""
346             if "SUMMARY" in text:
347                 summary = _parse_summary_lines(text.splitlines())
348                 if summary is not None:
349                     return summary
350     return None
351
352
353 def validate_against_summary(
354     statement: Statement, summary: StatementSummary, tol: float = 0.01
355 ) -> ValidationReport:
356     """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
357
358     Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
359     (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
360     error/omission, the Dr/Cr counts catch any extra/missing row, and the opening &
361     closing
362     balances catch a missing first/last row.
363     """
364     rows = [t.to_concept() for t in statement.transactions]
365     return verify(rows, summary, tol)
366
367 def write_csv(statement: Statement, out_path: Path | str) -> Path:
368     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
369
370     Writes real data ? call only for a local, gitignored destination.
371     """
372     out = Path(out_path)

```

```

373 out.parent.mkdir(parents=True, exist_ok=True)
374 with open(out, "w", newline="", encoding="utf-8") as fh:
375     writer = csv.writer(fh)
376     writer.writerow(
377         [
378             "date",
379             "value_date",
380             "narration",
381             "ref",
382             "withdrawal",
383             "deposit",
384             "closing_balance",
385             "page",
386         ]
387     )
388     for t in statement.transactions:
389         writer.writerow(
390             [
391                 t.date,
392                 t.value_date or "",
393                 t.narration,
394                 t.ref or "",
395                 f"{t.withdrawal:.2f}",
396                 f"{t.deposit:.2f}",
397                 f"{t.closing_balance:.2f}",
398                 t.page,
399             ]
400         )
401     return out
402
403
404 @register_parser
405 class HDFCParser(StatementParser):
406     """Registered parser for HDFC savings / transaction-account statements."""
407
408     def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
409         low = (text or "").lower()
410         is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
"".lower())
411         is_statement = "savings account details" in low or (
412             "narration" in low and "closing balance" in low
413         )
414         return is_hdfc and is_statement
415
416     def parse_and_insert(
417         self,
418         pdf_path: Path,
419         conn,
420         doc_id: int,
421         password: str | None = None,
422     ) -> int:
423         """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
424
425         Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
426         balance walk (and the bank's SUMMARY when present) so a downstream consumer
427         can tell a trustworthy parse from a suspect one. Returns the row count.
428         """
429         try:
430             stmt = parse_statement(pdf_path, password=password or "")
431         except ParseError:
432             raise
433         except Exception as exc:
434             raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc

```



```

435         for t in stmt.transactions:
436             dbmod.insert_hdfc_transaction(
437                 conn,
438                 document_id=doc_id,
439                 account_number=stmt.account_number,
440                 txn_date=t.date,
441                 value_date=t.value_date,
442                 particulars=t.narration,
443                 ref=t.ref,
444                 withdrawal=t.withdrawal,
445                 deposit=t.deposit,
446                 closing_balance=t.closing_balance,
447             )
448             summary = parse_summary(pdf_path, password=password or "")
449             ok = (
450                 validate_against_summary(stmt, summary).ok
451                 if summary is not None
452                 else reconcile(stmt).ok
453             )
454             verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
455             dbmod.record_parse(
456                 conn, doc_id, confidence=verdict.value, parser="HDFCParser",
457                 parser_version=__version__
458             )
459             return len(stmt.transactions)

```

3. user (2026-06-10T16:28:31.434Z)

```

src\muneem\__init__.py
src\muneem\config.py
src\muneem\offers.py
src\muneem\ocr.py
src\muneem\ingestion\__init__.py
src\muneem\ingestion\base.py
src\muneem\ingestion\gmail.py
src\muneem\parsers\__init__.py
src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\extract.py
src\muneem\parsers\savings.py
src\muneem\log.py
src\muneem\redaction.py
src\muneem\password_rules.py
src\muneem\secrets.py
src\muneem\statement_unlock.py
src\muneem\parsers\engine.py
src\muneem\unlock.py
src\muneem\db_key.py
src\muneem\parsers\base.py
src\muneem\ingestion\imap.py
src\muneem\ingestion\outlook.py
src\muneem\cli.py
src\muneem\recovery.py
src\muneem\errors.py
src\muneem\db.py
src\muneem\migrations.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py

```

4. user (2026-06-10T16:28:38.625Z)

```

1      """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3      This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4      (accounts, transactions, holdings, merchants, items) grows with later milestones.
5      ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6      ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7      """
8
9      from __future__ import annotations
10
11      import os
12      import sqlite3
13      from collections.abc import Iterator
14      from contextlib import contextmanager
15      from pathlib import Path
16
17      from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19      SCHEMA_VERSION = 4
20
21      # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22      # file does not, so we can tell them apart without a key.
23      _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25      _SCHEMA = """
26      CREATE TABLE IF NOT EXISTS documents (
27          id            INTEGER PRIMARY KEY,
28          source        TEXT NOT NULL,
29          external_id   TEXT,
30          sender        TEXT,
31          subject       TEXT,
32          received_date TEXT,
33          doc_type      TEXT,
34          sha256        TEXT NOT NULL UNIQUE,
35          path          TEXT,
36          status        TEXT NOT NULL DEFAULT 'parsed',
37          created_at    TEXT NOT NULL DEFAULT (datetime('now'))
38      );
39
40      CREATE TABLE IF NOT EXISTS offers (
41          id            INTEGER PRIMARY KEY,
42          document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43          merchant      TEXT,
44          promo_code    TEXT,
45          description   TEXT,
46          expiry_raw    TEXT,
47          expiry_type   TEXT,
48          source        TEXT NOT NULL DEFAULT 'text',
49          redeemed      INTEGER NOT NULL DEFAULT 0,
50          source_text   TEXT
51      );
52
53      CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55      -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56      -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57      -- Normalization into a unified view layer is planned for a later phase
58      -- once all bank schemas are stabilized.
59      CREATE TABLE IF NOT EXISTS axis_transactions (
60          id INTEGER PRIMARY KEY,
61          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62          account_number TEXT,
63          txn_date TEXT,

```

```

64         value_date TEXT,
65         particulars TEXT,
66         debit REAL,
67         credit REAL,
68         balance REAL
69     );
70
71     CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72         id INTEGER PRIMARY KEY,
73         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74         account_number TEXT,
75         txn_date TEXT,
76         particulars TEXT,
77         merchant_category TEXT,
78         amount REAL,
79         type TEXT
80     );
81
82     CREATE TABLE IF NOT EXISTS au_transactions (
83         id INTEGER PRIMARY KEY,
84         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85         account_number TEXT,
86         txn_date TEXT,
87         value_date TEXT,
88         particulars TEXT,
89         debit REAL,
90         credit REAL,
91         balance REAL
92     );
93
94     CREATE TABLE IF NOT EXISTS hdfc_transactions (
95         id INTEGER PRIMARY KEY,
96         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97         account_number TEXT,
98         txn_date TEXT,
99         value_date TEXT,
100        particulars TEXT,
101        ref TEXT,
102        withdrawal REAL,
103        deposit REAL,
104        closing_balance REAL
105    );
106    """
107
108
109    def _raw_key_pragma(key: str) -> str:
110        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111        return f"PRAGMA key = \"x'{key}'\""
112
113
114    def is_plaintext_sqlite(db_path: Path | str) -> bool:
115        """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118        without needing the key.
119        """
120        try:
121            with open(db_path, "rb") as fh:
122                return fh.read(16) == _SQLITE_MAGIC
123        except OSError:
124            return False
125
126
127    def _harden_permissions(path: str) -> None:

```

```

128         """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
129         ``:memory:``."""
130         if path == ":memory:":
131             return
132         try:
133             os.chmod(path, 0o600)
134         except OSError:
135             pass
136
137     def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138         """Open an encrypted connection via SQLCipher, verifying the key before returning.
139
140         A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
141         the
142         key is wrong (or the file is not an encrypted muneem ledger), this raises.
143         """
144         try:
145             import sqlcipher3
146         except ImportError as exc: # pragma: no cover - exercised only without the binding
147             raise EncryptedDatabaseError(
148                 "database encryption requested but the 'sqlcipher3' package is not
149                 installed"
150             ) from exc
151
152         conn = sqlcipher3.connect(path)
153         conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
154         try:
155             conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
156         except sqlcipher3.DatabaseError as exc:
157             conn.close()
158             raise EncryptedDatabaseError(
159                 f"could not open encrypted database {Path(path).name}: wrong key, or not an
160                 "
161                 "encrypted muneem ledger"
162             ) from exc
163
164         conn.row_factory = sqlcipher3.Row
165         return conn
166
167     @contextmanager
168     def connect(db_path: Path | str, *, key: str | None = None) ->
169     Iterator[sqlite3.Connection]:
170         """Open a connection (commit on success, always close). Parent dirs created.
171
172         With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
173         SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
174         path for
175         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
176         :func:`_open_encrypted`); the production ledger is opened via
177         :func:`connect_encrypted`.
178         """
179         path = str(db_path)
180         if path != ":memory:":
181             Path(path).parent.mkdir(parents=True, exist_ok=True)
182         if key is not None:
183             conn = _open_encrypted(path, key)
184             _harden_permissions(path)
185         else:
186             conn = sqlite3.connect(path)
187             conn.row_factory = sqlite3.Row
188             conn.execute("PRAGMA foreign_keys = ON")
189         try:
190             yield conn
191             conn.commit()

```

```

186         finally:
187             conn.close()
188
189
190     @contextmanager
191     def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
192         """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194         The production path: resolves (or mints, on first use) the 256-bit key from the OS
195         keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
196         SQLite
197         file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
198         "file is not a database" error.
199         """
200         from muneem.db_key import get_or_create_key
201
202         path = str(db_path)
203         if is_plaintext_sqlite(path):
204             raise EncryptedDatabaseError(
205                 f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
206 encrypt' "
207                 "to migrate it to an encrypted ledger"
208             )
209         with connect(path, key=get_or_create_key()) as conn:
210             yield conn
211
212     def encrypt_database(db_path: Path | str, key: str) -> int:
213         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
214         ``sqlcipher_export``.
215
216         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
217         then
218         securely delete (it still holds the cleartext data). Returns the number of user
219         tables
220         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
221         SQLite file.
222         """
223         import sqlcipher3
224
225         path = Path(db_path)
226         if not is_plaintext_sqlite(path):
227             raise EncryptedDatabaseError(
228                 f"{path.name} is not a plaintext SQLite database (already encrypted or
229 missing)"
230             )
231
232         enc_tmp = path.with_name(path.name + ".enc.tmp")
233         enc_tmp.unlink(missing_ok=True)
234         try:
235             src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
236             try:
237                 src_tables = src.execute(
238                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
239                 ).fetchone()[0]
240                 target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
241                 slashes
242                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY '{key}'")
243                 src.execute("SELECT sqlcipher_export('encrypted')")
244                 src.execute("DETACH DATABASE encrypted")
245             finally:
246                 src.close()
247
248         # Verify the encrypted copy BEFORE the destructive swap: it must open with the
249         key, pass

```

```

242         # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243         # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244         with connect(str(enc_tmp), key=key) as check:
245             integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246             ntables = check.execute(
247                 "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248             ).fetchone()[0]
249         if integrity != "ok" or ntables != src_tables:
250             raise EncryptedDatabaseError(
251                 f"encryption migration verification failed for {path.name} "
252                 f"({integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253             )
254
255         backup = path.with_name(path.name + ".plaintext.bak")
256         backup.unlink(missing_ok=True)
257         path.rename(backup)
258         enc_tmp.rename(path) # consumed on success
259         _harden_permissions(str(path))
260     finally:
261         # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262         enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = \"x'{new_key}'\"")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:

```

```

299         target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300         source.backup(target) # consistent snapshot of the live DB
301         finally:
302             target.close()
303     finally:
304         source.close()
305     _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()
321
322
323     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
325         cols = ", ".join(fields)
326         placeholders = ", ".join("?" for _ in fields)
327         cur = conn.execute(
328             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
329             tuple(fields.values()),
330         )
331         rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
checker
332         if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
333             raise RuntimeError(f"INSERT into {table} did not return a row id")
334         return int(rowid)
335
336
337     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
338         """Insert into documents table. Delegates to the shared helper for DRY."""
339         return _insert_row(conn, "documents", dict(fields))
340
341
342     def record_provenance(
343         conn: sqlite3.Connection,
344         *,
345         document_id: int,
346         source_type: str,
347         parser: str | None = None,
348         parser_version: str | None = None,
349         mapping_layer: str | None = None,
350         confidence: str | None = None,
351     ) -> None:
352         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
353
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)`, so
a single

```

```

355     provenance record per document is reachable by join. Idempotent on ``document_id``
(a re-parse
356     overwrites rather than duplicating).
357     """
358     conn.execute(
359         "INSERT INTO provenance "
360         "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
361         "VALUES (?, ?, ?, ?, ?, ?) "
362         "ON CONFLICT(document_id) DO UPDATE SET "
363         "source_type=excluded.source_type, parser=excluded.parser, "
364         "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365         "confidence=excluded.confidence",
366         (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367     )
368
369
370     def record_parse(
371         conn: sqlite3.Connection,
372         document_id: int,
373         *,
374         confidence: str,
375         parser: str,
376         source_type: str = "pdf_statement",
377         parser_version: str | None = None,
378         mapping_layer: str = "deterministic",
379     ) -> None:
380         """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
381         conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
document_id))
382         record_provenance(
383             conn,
384             document_id=document_id,
385             source_type=source_type,
386             parser=parser,
387             parser_version=parser_version,
388             mapping_layer=mapping_layer,
389             confidence=confidence,
390         )
391
392
393     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
394         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()
395
396
397     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
398         return _insert_row(conn, "offers", fields)
399
400
401     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
402         return conn.execute(
403             """
404             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
405                    d.external_id, d.path
406             FROM offers o
407             JOIN documents d ON d.id = o.document_id
408             ORDER BY o.id
409             """
410         ).fetchall()
411
412
413     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
414         return _insert_row(conn, "axis_transactions", fields)
415

```



```

416
417     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
418         return _insert_row(conn, "axis_cc_transactions", fields)
419
420
421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

5. user (2026-06-10T16:28:39.533Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (300). The file has 202 lines.</system-reminder>

6. user (2026-06-10T16:28:44.579Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
Axis specifics
6     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
mapping.
7
8     Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
reconcile
9     (parse/persist split + count only). No statement content is logged here.
10    """
11
12    from __future__ import annotations

```

```

13
14     import re
15     import sqlite3
16     from dataclasses import dataclass
17     from pathlib import Path
18
19     import pdfplumber
20
21     from muneem import __version__
22     from muneem import db as dbmod
23     from muneem.errors import ParseError
24     from muneem.redaction import redact_filename
25
26     from . import register_parser
27     from .base import StatementParser
28     from .profiles import AXIS_PROFILE
29     from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
30     from .validation import Confidence, parse_amount
31
32     _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
33     _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
34
35
36     def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
37         for rx in patterns:
38             m = rx.search(text)
39             if m:
40                 return m.group(1).strip()
41         return None
42
43
44     # --- Savings / transaction account (concept core)
-----
45
46
47     # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48     AxisSavings = SavingsParse
49     parse_axis_summary = parse_statement_summary
50     _is_txn_line = txn_line_predicate(AXIS_PROFILE)
51
52
53     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54         """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55         return parse_savings(pdf_path, AXIS_PROFILE, password)
56
57
58     @register_parser
59     class AxisParser(StatementParser):
60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66         def parse_and_insert(
67             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68         ) -> int:
69             """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70

```

```

71         Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72         parser can't yet read surfaces loudly instead of saving mislabeled rows.
73         """
74         try:
75             parsed = parse_axis_savings(pdf_path, password)
76         except ParseError:
77             raise
78         except Exception as exc:
79             # Filename only ? never statement text; original cause kept via `from`.
80             name = redact_filename(pdf_path)
81             raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83         if not parsed.verified:
84             dbmod.record_parse(
85                 conn,
86                 doc_id,
87                 confidence=Confidence.UNRECONCILED.value,
88                 parser="AxisParser",
89                 parser_version=__version__,
90             )
91             return 0
92
93         for r in parsed.rows:
94             dbmod.insert_axis_transaction(
95                 conn,
96                 document_id=doc_id,
97                 account_number=parsed.account_number,
98                 txn_date=r.txn_date,
99                 value_date=r.value_date or "",
100                 particulars=r.narration,
101                 debit=r.debit,
102                 credit=r.credit,
103                 balance=r.balance,
104             )
105             dbmod.record_parse(
106                 conn,
107                 doc_id,
108                 confidence=Confidence.RECONCILED.value,
109                 parser="AxisParser",
110                 parser_version=__version__,
111             )
112         return len(parsed.rows)
113
114
115     # --- Credit card (no running balance; count only)
-----
116
117
118     @dataclass
119     class AxisCCTxn:
120         txn_date: str
121         particulars: str
122         merchant_category: str
123         amount: float
124         type: str # "Cr" or "Dr"
125
126
127     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129         rows: list[AxisCCTxn] = []
130         for table in tables or []:
131             for row in table or []:
132                 if not row or len(row) < 9:
133                     continue

```

```

134         date_str = str(row[0]).strip() if row[0] else ""
135         if not _CC_DATE_RE.match(date_str):
136             continue
137         amount_str = str(row[8]).strip() if row[8] else ""
138         is_cr = "Cr" in amount_str
139         amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
140         rows.append(
141             AxisCCTxn(
142                 txn_date=date_str,
143                 particulars=str(row[2]).strip() if row[2] else "",
144                 merchant_category=str(row[7]).strip() if row[7] else "",
145                 amount=amount,
146                 type="Cr" if is_cr else "Dr",
147             )
148         )
149     return rows
150
151
152     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153         """Open an Axis credit-card statement and return (card_number, transactions)."""
154         card_number = "UNKNOWN"
155         rows: list[AxisCCTxn] = []
156         with pdfplumber.open(pdf_path, password=password) as pdf:
157             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
158             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159             for page in pdf.pages:
160                 rows.extend(_rows_from_cc_tables(page.extract_tables()))
161         return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text
168
169         def parse_and_insert(
170             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171         ) -> int:
172             try:
173                 card_number, rows = parse_axis_cc(pdf_path, password)
174             except ParseError:
175                 raise
176             except Exception as exc:
177                 raise ParseError(
178                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179                 ) from exc
180             for t in rows:
181                 dbmod.insert_axis_cc_transaction(
182                     conn,
183                     document_id=doc_id,
184                     account_number=card_number,
185                     txn_date=t.txn_date,
186                     particulars=t.particulars,
187                     merchant_category=t.merchant_category,
188                     amount=t.amount,
189                     type=t.type,
190                 )
191             # No running balance to reconcile; documents.status stays as the caller set it.
192             # record provenance (with no confidence verdict).
193             dbmod.record_provenance(

```

```

194         conn,
195         document_id=doc_id,
196         source_type="pdf_statement",
197         parser="AxisCreditCardParser",
198         parser_version=__version__,
199         mapping_layer="deterministic",
200     )
201     return len(rows)
202

```

7. user (2026-06-10T16:28:44.698Z)

```

1     """AU Small Finance Bank savings / transaction-account statement parser.
2
3     Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
5     gated** so a
6     vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
7     entirely in
8     ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
9     here.
10
11     """
12     from __future__ import annotations
13
14     import sqlite3
15     from pathlib import Path
16
17     from muneem import __version__
18     from muneem import db as dbmod
19     from muneem.errors import ParseError
20     from muneem.redaction import redact_filename
21
22     from . import register_parser
23     from .base import StatementParser
24     from .profiles import AUBANK_PROFILE
25     from .savings import SavingsParse, parse_savings
26     from .validation import Confidence
27
28     def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
29         """Parse an AU Bank savings statement via the shared engine-driven path."""
30         return parse_savings(pdf_path, AUBANK_PROFILE, password)
31
32     @register_parser
33     class AUParser(StatementParser):
34         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
35         None) -> bool:
36             if sender and "aubank" in sender.lower():
37                 return True
38             low = (text or "").lower()
39             return "au small finance bank" in low or "aubank" in low
40
41         def parse_and_insert(
42             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
43             None = None
44         ) -> int:
45             """Persist a savings parse **only if it reconciles**; otherwise store
46             nothing."""
47             try:
48                 parsed = parse_au_savings(pdf_path, password)
49             except ParseError:
50                 raise
51             except Exception as exc:

```

```

48         raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc
49
50     if not parsed.verified:
51         dbmod.record_parse(
52             conn,
53             doc_id,
54             confidence=Confidence.UNRECONCILED.value,
55             parser="AUParser",
56             parser_version=__version__,
57         )
58     return 0
59
60     for r in parsed.rows:
61         dbmod.insert_au_transaction(
62             conn,
63             document_id=doc_id,
64             account_number=parsed.account_number,
65             txn_date=r.txn_date,
66             value_date=r.value_date or "",
67             particulars=r.narration,
68             debit=r.debit,
69             credit=r.credit,
70             balance=r.balance,
71         )
72     dbmod.record_parse(
73         conn,
74         doc_id,
75         confidence=Confidence.RECONCILED.value,
76         parser="AUParser",
77         parser_version=__version__,
78     )
79     return len(parsed.rows)
80

```

8. user (2026-06-10T16:28:57.693Z)

```

1     """The ``muneem`` command-line interface.
2
3     Track A ships the promo spine: ``ingest`` a PDF into the local store and
4     ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5     codes rasterized into banners (approximate; flagged for verification). Pipeline
6     subcommands (``fetch``, ``txns``) land with their milestones.
7     """
8
9     from __future__ import annotations
10
11     from contextlib import contextmanager
12     from pathlib import Path
13
14     import typer
15
16     from muneem import __version__
17     from muneem import db as dbmod
18     from muneem.config import load_config
19     from muneem.log import get_logger
20     from muneem.redaction import redact_filename
21
22     _log = get_logger(__name__)
23
24     app = typer.Typer(
25         add_completion=False,
26         help="muneem ? a local-first personal-finance bookkeeper.",
27         no_args_is_help=True,
28     )

```

```

29 offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
30 app.add_typer(offered_app, name="offers")
31
32 txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33 app.add_typer(txns_app, name="txns")
34
35 db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36 app.add_typer(db_app, name="db")
37
38
39 @contextmanager
40 def _ledger(db_path: Path):
41     """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
CLI
42     error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
43     try:
44         with dbmod.connect_encrypted(db_path) as conn:
45             yield conn
46     except dbmod.EncryptedDatabaseError as exc:
47         typer.echo(str(exc), err=True)
48         raise typer.Exit(1) from exc
49
50
51 # Dispatch table for supported banks (extensible as new parsers land).
52 BANK_DISPATCH = {
53     "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54     "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55     "au": (dbmod.list_au_transactions, "debit", "credit"),
56     "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57 }
58
59
60 def _version_callback(value: bool) -> None:
61     if value:
62         typer.echo(f"muneem {__version__}")
63         raise typer.Exit()
64
65
66 @app.callback()
67 def main(
68     version: bool = typer.Option(
69         False,
70         "--version",
71         callback=_version_callback,
72         is_eager=True,
73         help="Show the muneem version and exit.",
74     ),
75 ) -> None:
76     """muneem command-line interface."""
77
78
79 @app.command()
80 def info() -> None:
81     """Show resolved local paths (no secrets, no sensitive data)."""
82     cfg = load_config()
83     present = "present" if cfg.config_file.is_file() else "absent"
84     typer.echo(f"muneem {__version__}")
85     typer.echo(f"config dir : {cfg.config_dir}")
86     typer.echo(f"data dir   : {cfg.data_dir}")
87     typer.echo(f"db path    : {cfg.db_path}")
88     typer.echo(f"staging   : {cfg.staging_dir}")
89     typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92 def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:

```

```

bool) -> None:
93     """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
94
95     The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
extraction,
96     since the text needed to pick a parser is itself encrypted. The resolved password is
forwarded
97     to extraction and the parser, used in-memory only ? never logged.
98     """
99     from muneem import db as dbmod
100    from muneem.extract import extract
101    from muneem.parsers import get_parser
102    from muneem.statement_unlock import resolve_password
103    from muneem.unlock import is_encrypted
104
105    password: str | None = ""
106    if is_encrypted(pdf):
107        password = resolve_password(pdf, interactive=interactive)
108        if password is None:
109            hint = " (run without --no-prompt to enter it)" if not interactive else ""
110            typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
111            raise typer.Exit(1)
112
113    ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
str
114    parser = get_parser(ex.text)
115    if parser is None:
116        typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
117        raise typer.Exit(1)
118
119    with _ledger(db_path) as conn:
120        dbmod.init_db(conn)
121        existing = dbmod.get_document_by_sha(conn, ex.sha256)
122        if existing is not None:
123            typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
124            return
125        doc_id = dbmod.insert_document(
126            conn,
127            source="file",
128            external_id=pdf.name,
129            doc_type=doc_type,
130            sha256=ex.sha256,
131            path=str(pdf.resolve()),
132            status="parsed",
133        )
134        count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
135        doc = dbmod.get_document_by_sha(conn, ex.sha256)
136        status = doc["status"] if doc else "parsed"
137
138        typer.echo(
139            f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
140            f"via {parser.__class__.__name__} [{status}]"
141        )
142
143
144    def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
145        """Run one statement through the full parse pipeline into an in-memory DB.
146
147        Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
status
148        (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
149        Nothing is persisted and no statement value is surfaced ? only the verdict and

```



```

counts.
150     """
151     from muneem import db as dbmod
152     from muneem.extract import extract
153     from muneem.parsers import get_parser
154     from muneem.statement_unlock import resolve_password
155     from muneem.unlock import is_encrypted
156
157     password = ""
158     if is_encrypted(pdf):
159         password = resolve_password(pdf, interactive=interactive) or ""
160         if not password:
161             return ("locked", 0, None)
162     try:
163         ex = extract(pdf, password=password)
164         parser = get_parser(ex.text)
165     except Exception as exc: # eval must not crash on one bad file; record the redacted
reason
166         _log.warning(
167             "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
168         )
169         return ("error", 0, None)
170     if parser is None:
171         return ("no-parser", 0, None)
172     name = parser.__class__.__name__
173     try:
174         with dbmod.connect(":memory:") as conn:
175             dbmod.init_db(conn)
176             doc_id = dbmod.insert_document(
177                 conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
178             )
179             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
180             doc = dbmod.get_document_by_sha(conn, ex.sha256)
181             status = doc["status"] if doc else "parsed"
182     except Exception as exc:
183         _log.warning(
184             "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
185         )
186         return ("error", 0, name)
187     return (status, count, name)
188
189
190 @app.command("eval")
191 def eval_statements(
192     folder: Path = typer.Argument(
193         ...,
194         exists=True,
195         file_okay=False,
196         readable=True,
197         help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198     ),
199     prompt: bool = typer.Option(
200         False,
201         "--prompt/--no-prompt",
202         help="Prompt for a password when cached/configured candidates don't unlock a
file.",
203     ),
204 ) -> None:
205     """Evaluate the parsers against a folder of statements and report the reconcile
rate.
206

```

```

207         The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
208         statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
209         an in-memory DB; only counts / statuses / parser names print ? never statement
values.
210         """
211         pdfs = sorted(folder.glob("*.pdf"))
212         if not pdfs:
213             typer.echo(f"no .pdf files in {folder}")
214             return
215         reconciled = 0
216         for pdf in pdfs:
217             outcome, count, name = _eval_statement(pdf, interactive=prompt)
218             reconciled += outcome == "reconciled"
219             typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")
220         typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
221
222
223     @app.command()
224     def ingest(
225         pdf: Path = typer.Argument(
226             ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
227         ),
228         doc_type: str = typer.Option("promo", help="Document category."),
229         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
230         ocr: bool = typer.Option(
231             False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
232         ),
233         no_prompt: bool = typer.Option(
234             False,
235             "--no-prompt",
236             help="Never prompt for a statement password; use only cached/configured
candidates.",
237         ),
238     ) -> None:
239         """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
240         from muneem.extract import extract
241         from muneem.offers import extract_offer
242
243         db_path = db or load_config().db_path
244
245         if doc_type == "bank_statement":
246             _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
247             return
248
249         from muneem import db as dbmod
250
251         ex = extract(pdf)
252         text_fields = extract_offer(ex.text)
253         # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
254         found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
255         expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
256
257         if ocr:
258             from muneem.ocr import ocr_pdf, tesseract_available
259
260             if not tesseract_available():
261                 typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
262             else:
263                 ocr_fields = extract_offer(ocr_pdf(pdf))
264                 known = {c for c, _ in found}
265                 for code in ocr_fields.codes:

```

```

266         if code not in known:
267             found.append((code, "ocr"))
268             known.add(code)
269         if expiry_raw is None:
270             expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
271
272     with _ledger(db_path) as conn:
273         dbmod.init_db(conn)
274         existing = dbmod.get_document_by_sha(conn, ex.sha256)
275         if existing is not None:
276             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
277             return
278         doc_id = dbmod.insert_document(
279             conn,
280             source="file",
281             external_id=pdf.name,
282             doc_type=doc_type,
283             sha256=ex.sha256,
284             path=str(pdf.resolve()),
285             status="parsed",
286         )
287         for code, src in found:
288             dbmod.insert_offer(
289                 conn,
290                 document_id=doc_id,
291                 promo_code=code,
292                 expiry_raw=expiry_raw,
293                 expiry_type=expiry_type,
294                 source=src,
295                 source_text=ex.text[:2000],
296             )
297
298     summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
299     typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
300
301
302 @offers_app.command("list")
303 def offers_list(
304     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
305     dir)."),
306     ) -> None:
307     """List extracted offers."""
308     from muneem import db as dbmod
309
310     db_path = db or load_config().db_path
311     if not Path(db_path).exists():
312         typer.echo("no offers yet (database not created).")
313         return
314
315     with _ledger(db_path) as conn:
316         dbmod.init_db(conn)
317         rows = dbmod.list_offers(conn)
318
319     if not rows:
320         typer.echo("no offers found.")
321         return
322
323     for row in rows:
324         who = row["merchant"] or row["external_id"] or "?"
325         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
326         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
327         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
328
329
330 @txns_app.command("list")
331 def txns_list(

```

```

330     bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,
axis)."),
331     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
332     limit: int = typer.Option(50, help="Max number of transactions to show."),
333 ) -> None:
334     """List parsed transactions for a specific bank."""
335     from muneem import db as dbmod
336
337     db_path = db or load_config().db_path
338     if not Path(db_path).exists():
339         typer.echo("database not created yet.")
340         return
341
342     bank = bank.lower()
343     if bank not in BANK_DISPATCH:
344         typer.echo(f"Unknown bank: {bank}", err=True)
345         return
346
347     list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
348     with _ledger(db_path) as conn:
349         dbmod.init_db(conn)
350         rows = list_fn(conn, limit=limit)
351
352     if not rows:
353         typer.echo(f"no {bank} transactions found.")
354         return
355     for row in rows:
356         if bank == "axis_cc":
357             amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358         else:
359             amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360         part = (row["particulars"] or "")[:50]
361         typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key      : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),

```

```

390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392         from muneem import db as dbmod
393         from muneem.db_key import get_or_create_key
394
395         db_path = db or load_config().db_path
396         if not Path(db_path).exists():
397             typer.echo(
398                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399             )
400             return
401         if not dbmod.is_plaintext_sqlite(db_path):
402             typer.echo(
403                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404             )
405             return
406         n = dbmod.encrypt_database(db_path, get_or_create_key())
407         typer.echo(
408             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
409             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410         )
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
415         from muneem import db as dbmod
416         from muneem.db_key import peek_key
417
418         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
419             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
err=True)
420             raise typer.Exit(1)
421         key = peek_key()
422         if not key:
423             typer.echo("no encryption key in the keychain for this ledger.", err=True)
424             raise typer.Exit(1)
425         return key
426
427
428     @db_app.command("rotate-key")
429     def db_rotate_key(
430         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
431     ) -> None:
432         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
433         from muneem import db as dbmod
434         from muneem import db_key
435
436         db_path = db or load_config().db_path
437         current = _require_encrypted_ledger(db_path)
438         fresh = db_key.new_key()
439         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
the file.
440         db_key.set_key(fresh)
441         try:
442             dbmod.rekey_database(db_path, current, fresh)
443         except dbmod.EncryptedDatabaseError as exc:
444             db_key.set_key(current) # roll back; the file is still under the old key
445             typer.echo(str(exc), err=True)
446             raise typer.Exit(1) from exc

```

```

447         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
448
449
450     @db_app.command("backup")
451     def db_backup(
452         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454     ) -> None:
455         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456         from muneem import db as dbmod
457
458         db_path = db or load_config().db_path
459         key = _require_encrypted_ledger(db_path)
460         try:
461             dbmod.backup_database(db_path, dest, key)
462         except dbmod.EncryptedDatabaseError as exc:
463             typer.echo(str(exc), err=True)
464             raise typer.Exit(1) from exc
465         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466
467
468     @db_app.command("setup-recovery")
469     def db_setup_recovery(
470         out: Path | None = typer.Option(
471             None, help="Recovery file path (default: <data dir>/muneem-recovery.json).",
472         ),
473     ) -> None:
474         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
477         file holds only the *wrapped* key ? useless without the passphrase.
478         """
479         import getpass
480
481         from muneem import recovery
482         from muneem.db_key import peek_key
483
484         key = peek_key()
485         if not key:
486             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487             raise typer.Exit(1)
488         out_path = out or (load_config().data_dir / "muneem-recovery.json")
489         passphrase = getpass.getpass("New recovery passphrase: ")
490         if passphrase != getpass.getpass("Confirm passphrase: "):
491             typer.echo("passphrases did not match.", err=True)
492             raise typer.Exit(1)
493         try:
494             recovery.write_recovery_file(out_path, key, passphrase)
495         except recovery.RecoveryError as exc:
496             typer.echo(str(exc), err=True)
497             raise typer.Exit(1) from exc
498         typer.echo(
499             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
/ "
500             "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501             "lost keychain means the ledger and its backups are unrecoverable."
502         )
503
504

```

```

505     @db_app.command("recover")
506     def db_recover(
507         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
508         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
509     ) -> None:
510         """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
511         import getpass
512
513         from muneem import db as dbmod
514         from muneem import db_key, recovery
515
516         passphrase = getpass.getpass("Recovery passphrase: ")
517         try:
518             key = recovery.read_recovery_file(file, passphrase)
519         except recovery.RecoveryError as exc:
520             typer.echo(str(exc), err=True)
521             raise typer.Exit(1) from exc
522         db_path = db or load_config().db_path
523         # If the ledger is present, verify the recovered key actually opens it before
committing it.
524         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
525             try:
526                 with dbmod.connect(db_path, key=key) as conn:
527                     conn.execute("SELECT 1")
528             except dbmod.EncryptedDatabaseError as exc:
529                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530                 raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537

```

9. assistant (2026-06-10T16:29:07.370Z)

The core claims are confirmed in the code. Now let me check for existing mitigations ? tests, docs, or any cleanup path that handles the unreconciled case.

10. user (2026-06-10T16:29:08.895Z)

```

tests\test_provenance.py:75: def
test_unreconciled_parse_records_unreconciled_provenance(monkeypatch):
tests\test_provenance.py:82:         assert dbmod.get_provenance(conn, doc)["confidence"] ==
"unreconciled"
tests\parsers\test_au.py:71:         assert dbmod.get_document_by_sha(conn, "au2")["status"] ==
"unreconciled"
tests\parsers\test_hdfc.py:197: def test_parse_and_insert_records_unreconciled(tmp_path,
monkeypatch):
tests\parsers\test_hdfc.py:210:         assert dbmod.get_document_by_sha(conn, "h2")["status"] ==
"unreconciled"
tests\parsers\test_local_au.py:24: def test_local_au_never_stores_unreconciled(pdf_path,
tmp_path):
tests\parsers\test_local_au.py:47:         f"{pdf_path.name}: stored {n} row(s) with
status={status} ? must not persist unreconciled"
tests\parsers\test_axis.py:76: def test_savings_refuses_when_unreconciled():
tests\parsers\test_axis.py:114:         assert dbmod.get_document_by_sha(conn, "ax2")["status"]
== "unreconciled"
tests\parsers\test_local_axis.py:26: def test_local_axis_never_stores_unreconciled(pdf_path,
tmp_path):

```

```
tests\parsers\test_local_axis.py:45:    # document unreconciled) or, if it stored rows, they
reconciled. Never garbage.
tests\parsers\test_local_axis.py:49:    "stop-gap must not persist unreconciled rows"
tests\parsers\test_validation.py:94:    assert not r.ok and r.confidence is
Confidence.UNRECONCILED
```

11. user (2026-06-10T16:29:09.843Z)

```
docs\ROADMAP.md:232:- Two fixtures run the **full pipeline** (`extract` ? `get_parser` ?
`parse_and_insert` ? assert
tests\test_cli.py:42:    def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
src\muneem\cli.py:134:    count = parser.parse_and_insert(pdf, conn, doc_id,
password=password)
src\muneem\cli.py:179:    count = parser.parse_and_insert(pdf, conn, doc_id,
password=password)
tests\parsers\test_au.py:45: def test_parse_and_insert_stores_when_verified(tmp_path,
monkeypatch):
tests\parsers\test_au.py:51:     n = AUParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
tests\parsers\test_au.py:62: def test_parse_and_insert_refuses_when_unverified(tmp_path,
monkeypatch):
tests\parsers\test_au.py:68:     n = AUParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
tests\parsers\test_axis.py:5: ``parse_and_insert``'s store-iff-reconciled gate is checked by
monkeypatching that function.
tests\parsers\test_axis.py:86: def test_parse_and_insert_stores_when_verified(tmp_path,
monkeypatch):
tests\parsers\test_axis.py:92:     n = AxisParser().parse_and_insert(Path("x.pdf"), conn,
doc_id)
tests\parsers\test_axis.py:103: def test_parse_and_insert_refuses_when_unverified(tmp_path,
monkeypatch):
tests\parsers\test_axis.py:111:     n = AxisParser().parse_and_insert(Path("x.pdf"), conn,
doc_id)
tests\test_provenance.py:59:     n = au.AUParser().parse_and_insert(Path("x.pdf"), conn, doc,
password=None)
tests\test_provenance.py:80:     n = au.AUParser().parse_and_insert(Path("x.pdf"), conn, doc,
password=None)
tests\test_provenance.py:90:     axis.AxisCreditCardParser().parse_and_insert(Path("cc.pdf"),
conn, doc, password=None)
tests\parsers\test_hdfc.py:4: geometry ? no real data, fast, offline. ``parse_and_insert`` is
tested by
tests\parsers\test_hdfc.py:176: def test_parse_and_insert_records_reconciled(tmp_path,
monkeypatch):
tests\parsers\test_hdfc.py:188:     n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"),
conn, doc_id)
tests\parsers\test_hdfc.py:197: def test_parse_and_insert_records_unreconciled(tmp_path,
monkeypatch):
tests\parsers\test_hdfc.py:209:     hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn,
doc_id)
tests\parsers\test_hdfc.py:244:     n = hdfc.HDFCParser().parse_and_insert(pdf_path, conn,
doc_id, password=pw)
src\muneem\parsers\axis.py:66:     def parse_and_insert(
src\muneem\parsers\axis.py:169:     def parse_and_insert(
tests\parsers\test_local_axis.py:40:     n = parser.parse_and_insert(pdf_path, conn, doc_id,
password=pw)
src\muneem\parsers\au.py:39:     def parse_and_insert(
src\muneem\parsers\base.py:31:     def parse_and_insert(
tests\parsers\test_local_au.py:42:     n = parser.parse_and_insert(pdf_path, conn, doc_id,
password=pw)
tests\test_regression_synth.py:4: ``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert``
? assert a **reconciled** parse. This
tests\test_regression_synth.py:98:     n = parser.parse_and_insert(pdf, conn, doc_id)
tests\parsers\test_parse_errors.py:40:     parser.parse_and_insert(pdf, conn, 1,
password="pw")
src\muneem\parsers\hdfc.py:416:     def parse_and_insert(
```

12. user (2026-06-10T16:29:16.638Z)


```

1      """Tests for the HDFC statement parser (muneem.parsers.hdfc).
2
3      Column logic is exercised on *synthetic* word lists that mimic the real statement
4      geometry ? no real data, fast, offline. ``parse_and_insert`` is tested by
5      monkeypatching the parse functions (not by mocking pdfplumber), and a
6      ``local_statements`` test runs the real parser on the user's PDFs (skipped in CI).
7      """
8
9      import os
10     from pathlib import Path
11
12     import pytest
13
14     from muneem import db as dbmod
15     from muneem.parsers import hdfc
16
17
18     def W(text, x0, top, x1=None):
19         """A minimal pdfplumber-style word dict."""
20         return {
21             "text": text,
22             "x0": float(x0),
23             "x1": float(x1 if x1 is not None else x0 + 5 * len(text)),
24             "top": float(top),
25             "bottom": float(top) + 8,
26         }
27
28
29     def _sample_words():
30         """A 3-transaction table whose running balance reconciles, with header, a
31         per-page summary box above it, wrapped narration, and a footer marker."""
32         words = []
33         words += [W("Opening", 37, 50), W("Balance", 72, 50), W("01/02/2026", 52, 60)]
34         words += [
35             W("Date", 72, 80),
36             W("Narration", 175, 80),
37             W("Withdrawals", 296, 80),
38             W("Deposits", 386, 80),
39             W("Closing", 473, 80),
40             W("Balance", 504, 80),
41         ]
42         words += [
43             W("05/02/2026", 52, 100, 92),
44             W("UPI-SWIGGY-PAY", 111, 100),
45             W("500.00", 330, 100, 358),
46             W("0.00", 428, 100, 443),
47             W("9,500.00", 519, 100, 561),
48         ]
49         words += [W("okhdfcbank-CONTINUED", 111, 109)]
50         words += [W("04/02/2026", 122, 118, 162), W("123456789012", 180, 118, 255)]
51         words += [
52             W("08/02/2026", 52, 140, 92),
53             W("SALARY-CREDIT", 111, 140),
54             W("0.00", 340, 140, 358),
55             W("2,000.00", 415, 140, 443),
56             W("11,500.00", 515, 140, 561),
57         ]
58         words += [
59             W("10/02/2026", 52, 160, 92),
60             W("ATM-WDL", 111, 160),
61             W("1,500.50", 320, 160, 358),
62             W("0.00", 428, 160, 443),
63             W("9,999.50", 519, 160, 561),
64         ]
65         words += [W("STATEMENT", 200, 180), W("SUMMARY", 260, 180)]

```

```

66         return words
67
68
69     def _parse_sample():
70         body = hdfc._table_body(hdfc._group_lines(_sample_words()))
71         return hdfc.extract_transactions_from_lines(body, page_no=3)
72
73
74     def _sample_statement():
75         return hdfc.Statement(
76             path=Path("x"), transactions=_parse_sample(), account_number="50100000000000"
77         )
78
79
80     def test_columns_and_continuation_assembly():
81         t1, t2, t3 = _parse_sample()
82         assert t1.date == "05/02/2026"
83         assert t1.withdrawal == 500.0 and t1.deposit == 0.0 and t1.closing_balance == 9500.0
84         assert t1.value_date == "04/02/2026"
85         assert t1.ref == "123456789012"
86         assert t1.narration == "UPI-SWIGGY-PAY okhdfcbank-CONTINUED 123456789012"
87         assert t1.page == 3
88         assert t2.deposit == 2000.0 and t2.closing_balance == 11500.0
89         assert t3.withdrawal == 1500.5 and t3.closing_balance == 9999.5
90
91
92     def test_summary_box_and_footer_excluded():
93         txns = _parse_sample()
94         assert [t.date for t in txns] == ["05/02/2026", "08/02/2026", "10/02/2026"]
95         assert "STATEMENT" not in txns[-1].narration and "SUMMARY" not in txns[-1].narration
96
97
98     def test_reconciliation():
99         assert hdfc.reconcile(_sample_statement()).ok
100         txns = _parse_sample()
101         txns[2].closing_balance = 9999.49 # off by a paisa
102         report = hdfc.reconcile(hdfc.Statement(path=Path("x"), transactions=txns))
103         assert report.breaks == 1 and report.first_break == 2 and not report.ok
104
105
106     def test_parse_summary_lines_and_validation():
107         lines = [
108             "SUMMARY",
109             "Opening Balance Debit Amount Credit Amount Closing Balance",
110             "10,000.00 2,000.50 2,000.00 9,999.50",
111             "Debit Count Credit Count",
112             "2 1",
113         ]
114         s = hdfc._parse_summary_lines(lines)
115         assert s.opening_balance == 10000.0 and s.closing_balance == 9999.5
116         assert s.debit_count == 2 and s.credit_count == 1
117         report = hdfc.validate_against_summary(_sample_statement(), s)
118         assert report.ok and report.debit_count_ok and report.credit_count_ok
119         assert report.opening_ok and report.closing_ok
120
121
122     def test_validation_catches_missing_last_row():
123         stmt = hdfc.Statement(path=Path("x"), transactions=_parse_sample()[::-1])
124         summary = hdfc.StatementSummary(
125             opening_balance=10000.0, closing_balance=9999.50, debit_count=2, credit_count=1
126         )
127         report = hdfc.validate_against_summary(stmt, summary)
128         assert report.reconciles and report.closing_ok is False and not report.ok
129
130

```

```

131     @pytest.mark.parametrize(
132         "token, expected",
133         [
134             ("1,23,456.78", 123456.78),
135             ("0.00", 0.0),
136             ("1,000.00Cr", 1000.0),
137             ("9,999.50Dr", 9999.50),
138             ("abc", None),
139             ("100", None),
140             ("10/02/2026", None),
141         ],
142     )
143     def test_to_amount(token, expected):
144         assert hdfc._to_amount(token) == expected
145
146     def test_transaction_to_concept_maps_onto_canonical_vocabulary():
147         """The reference parser maps its native row onto the bank-agnostic concept
148         schema."""
149         t = hdfc.Transaction(
150             date="05/02/2026",
151             narration="UPI-SWIGGY",
152             withdrawal=500.0,
153             deposit=0.0,
154             closing_balance=9500.0,
155             value_date="04/02/2026",
156             ref="123456789012",
157             page=3,
158         )
159         c = t.to_concept()
160         assert c.txn_date == "05/02/2026" and c.narration == "UPI-SWIGGY"
161         assert c.debit == 500.0 and c.credit == 0.0 and c.balance == 9500.0
162         assert c.value_date == "04/02/2026" and c.reference == "123456789012"
163
164     def test_can_handle():
165         parser = hdfc.HDFCParser()
166         statement = "HDFC BANK Savings Account Details Narration Closing Balance"
167         assert parser.can_handle(statement)
168         assert parser.can_handle(statement, sender="alerts@hdfcbank.net")
169         # sender alone, with no statement markers, must NOT match (e.g. a promo from HDFC)
170         assert not parser.can_handle("anything", sender="alerts@hdfcbank.net")
171         assert not parser.can_handle("HDFC Bank promo: use code SAVE50")
172         assert not parser.can_handle("Axis Bank Statement of Account Narration Closing
173         Balance")
174
175     def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
176         monkeypatch.setattr(hdfc, "parse_statement", lambda *a, **k: _sample_statement())
177         monkeypatch.setattr(
178             hdfc,
179             "parse_summary",
180             lambda *a, **k: hdfc.StatementSummary(
181                 opening_balance=10000.0, closing_balance=9999.50, debit_count=2,
182                 credit_count=1
183             ),
184         )
185         with dbmod.connect(tmp_path / "t.db") as conn:
186             dbmod.init_db(conn)
187             doc_id = dbmod.insert_document(conn, source="file", sha256="h1",
188                 status="parsed")
189             n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
190             assert n == 3
191             rows = dbmod.list_hdfc_transactions(conn, limit=100)
192             assert len(rows) == 3

```

```

192         assert rows[0]["account_number"] == "5010000000000000"
193         assert {r["particulars"] for r in rows} # narration mapped to particulars
194         assert dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
195
196
197     def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
198         broken = _parse_sample()
199         broken[2].closing_balance = 1.0 # break the balance walk
200         monkeypatch.setattr(
201             hdfc,
202             "parse_statement",
203             lambda *a, **k: hdfc.Statement(path=Path("x"), transactions=broken),
204         )
205         monkeypatch.setattr(hdfc, "parse_summary", lambda *a, **k: None) # fall back to
reconcile()
206         with dbmod.connect(tmp_path / "t.db") as conn:
207             dbmod.init_db(conn)
208             doc_id = dbmod.insert_document(conn, source="file", sha256="h2",
status="parsed")
209             hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
210             assert dbmod.get_document_by_sha(conn, "h2")["status"] == "unreconciled"
211
212
213     @pytest.mark.local_statements
214     @pytest.mark.parametrize(
215         "pdf_path",
216         sorted(Path("testdata/hdfc-statements").glob("*.pdf")),
217         ids=lambda p: p.name,
218     )
219     def test_real_hdfc_statement_validates(pdf_path, tmp_path):
220         """Local-only: parse the real statements and require the parse to agree with
221         HDFC's own SUMMARY (Dr/Cr counts + opening/closing) and reconcile fully.
222
223         Set MUNEEEM_TEST_HDFC_PW; never runs in CI. Surfaces counts/booleans only.
224         """
225         pw = os.environ.get("MUNEEEM_TEST_HDFC_PW")
226         if not pw:
227             pytest.skip("needs MUNEEEM_TEST_HDFC_PW + real statements (never in CI)")
228
229         stmt = hdfc.parse_statement(pdf_path, password=pw)
230         assert stmt.transactions, f"{pdf_path.name}: no transactions parsed"
231         summary = hdfc.parse_summary(pdf_path, password=pw)
232         assert summary is not None, f"{pdf_path.name}: SUMMARY block not found"
233         report = hdfc.validate_against_summary(stmt, summary)
234         assert report.ok, (
235             f"{pdf_path.name}: parse disagrees with bank SUMMARY ?
reconciles={report.reconciles} "
236             f"dr={report.debit_count_ok} cr={report.credit_count_ok} "
237             f"open={report.opening_ok} close={report.closing_ok}"
238         )
239
240         # End-to-end through the registered parser + DB, incl. the status it records.
241         with dbmod.connect(tmp_path / "t.db") as conn:
242             dbmod.init_db(conn)
243             doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
244             n = hdfc.HDFCParser().parse_and_insert(pdf_path, conn, doc_id, password=pw)
245             assert n == len(stmt.transactions)
246             assert dbmod.get_document_by_sha(conn, pdf_path.name)["status"] == "reconciled"
247

```

13. user (2026-06-10T16:29:58.000Z)

docs\feedbackJune2026.md:42:license allowlist, ****mypy**** + ****ruff**** + ****80% coverage gate**** in CI and the pre-push hook (#38/#40),

docs\feedbackJune2026.md:117:Decision stands (DeepSeek-OCR-3B, MIT, 4-bit+SDPA on the 2070S, reconcile-gated, self-consistency)

docs\feedbackJune2026.md:121:synthetic now; cloud eval-only, approval-gated) extend `muneem eval`. **Phase E (after D.3).**

docs\feedbackJune2026.md:126:experimentally, without per-session approval + redaction + a test gate. Merchant canon (\$8.5) is

docs\feedbackJune2026.md:133:`schema\_migrations` table, each with an **idempotency/up-checksum**, gated by the `user\_version`

docs\feedbackJune2026.md:253: **true on-device, self-custody-key decryption** (the key gate); CC/bonds go live on AA; a free

docs\encryption-at-rest.md:5:muneem aggregates the user's entire financial life into one local SQLite file ? a honeypot.

docs\CURRENT_STATE.md:29:[Omitted long matching line]

docs\CURRENT_STATE.md:30:[Omitted long matching line]

docs\CURRENT_STATE.md:49:* **Local git hooks** (`githubhooks/`, enable via `make hooks`): `pre-commit` blocks staging sensitive files; `pre-push` runs ruff + the offline suite (incl. the invariant tripwires) so only green code reaches GitHub. The pre-push gate is the **local** stand-in for server-side branch protection, which GitHub gates behind Pro / public repos (muneem stays private). Both are bypassable with `--no-verify` (the conscious "explicitly allowed" escape).

docs\CURRENT_STATE.md:53:Concept-core **steps 123 are done**, and **password unlock is now wired into `muneem ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 6/6, AU 4/4, Axis 6/7** (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:

docs\DEPENDENCIES.md:17: (cloud LLMs, paid OCR/vision) are a **fallback only**, gated behind explicit approval.

docs\DEPENDENCIES.md:43:| pytest-cov *(dev)* | MIT | coverage measurement + the >80% CI gate (coverage.py is Apache-2.0) |

docs\DEPENDENCIES.md:45:| mypy *(dev)* | MIT | static type checking (CI + pre-push gate) |

docs\onboarding-a-bank.md:28:It prints, per file, the outcome (`reconciled` / `unreconciled` / `locked` / `no-parser` / `error`),

docs\onboarding-a-bank.md:34:> mislabeled rows. Diagnose with the (masked) clustering tools; the `unreconciled` files are safe.

docs\onboarding-a-bank.md:40:folder is the quick local gate.

docs\onboarding-a-bank.md:42:### 3. Synthetic fixture ? gate the parser in cloud CI

docs\onboarding-a-bank.md:46:runs on every push (Layer-1), so the parser is regression-gated without exposing statements.

docs\ingesting-statements.md:27:`[reconciled]` means the parse is trusted; `[unreconciled]` means it couldn't be verified, so

docs\ingesting-statements.md:80:- Transaction parses are **reconciliation-gated**: an unverifiable parse stores nothing rather than

docs\ingesting-statements.md:89:| `? [unreconciled]` (0 transactions) | The parser couldn't verify the parse (e.g. an Axis vintage `extract\_tables` mangles); nothing was stored. See [parser-architecture.md](./parser-architecture.md). |

docs\research-open-banking-india.md:14:**regulatory gate**: only an entity *registered with and regulated by* RBI/SEBI/IRDAI/PFRDA can be a

docs\research-open-banking-india.md:119:is *impossible* today (regulatory gate) and *undesirable* (mandatory paid cloud tier that decrypts

docs\research-open-banking-india.md:136: confirmed" to "available." *The single most important gate for muneem's privacy model.*

docs\research-open-banking-india.md:145:RBI Master Direction on NBFC-AA + **Sahamati FIU FAQ** (the regulatory gate). **ReBIT NBFC-AA API

docs\prior-art-email-mining.md:90:- **Security as a first-class constraint.** Email-Mining was a coupon-finder; the worst-case leak was a expired DataQuest code. muneem aggregates the user's full financial life ? the threat model is categorically different.

docs\parser-architecture.md:6:> (MIT)**, with **Qwen2.5-VL-7B (Apache-2.0)** as a heavier fallback, reconcile-gated (\$4, \$7) ? and on

docs\parser-architecture.md:61:## 4. The extraction ladder (cheapest first; every layer gated by the verifier)

docs\parser-architecture.md:110:- reconciliation breaks, or parsed counts ? printed summary ? **quarantine** (store nothing or store

docs\parser-architecture.md:111: flagged; the oracle-gated parsers already refuse to persist unreconciled rows)

docs\parser-architecture.md:119:reconcile identity**, so the verifier rejects them. L3 traps therefore: gate every VLM parse on

docs\parser-architecture.md:158: oracle (per-row walk, else total reconcile), oracle-gated.
Real-data precision (local): ****AU 4/4****
docs\parser-architecture.md:160: mangles, so they store nothing (the safety invariant holds).
HDFC stays bespoke clustering.
docs\parser-architecture.md:180:- ****Cloud-golden (eval-only, approval-gated, off by default):****
for a layout no local method can
docs\ROADMAP.md:30:> (per-row walk, else total reconcile), oracle-gated. See [parser-
architecture.md](./parser-architecture.md)
docs\ROADMAP.md:49: ledger; oracle-gated, stores nothing). June 2025 recovered via balance-
delta. ? "?? Known issues".
docs\ROADMAP.md:50:3. ****L3 local-vision fallback**** ? DeepSeek-OCR-3B (MIT), reconcile-gated; dev
RTX 2070S ? 4-bit + SDPA.
docs\ROADMAP.md:52:4. ****Golden eval sets**** ? local-golden + synthetic-golden; cloud-golden eval-
only & approval-gated.
docs\ROADMAP.md:153:- [x] ****A.3 (M)**** Promo extraction, ****deterministic-first****:
regex/heuristics for promo codes + expiry dates over the unified text. Where deterministic
extraction misses, an ****LLM fallback**** is permitted **because promo is non-sensitive** ? but that
pulls in a cloud service, so it's gated on a quick okay (reuse Email-Mining's Gemini path, go
local via Ollama, or use Anthropic ? your call when we get there).
docs\ROADMAP.md:242: gate even without a PR.
docs\ROADMAP.md:254: oracle-gated safety + reconcile (`test\_hdfc.py`, `test\_local\_axis.py`,
`test\_local\_au.py`).
docs\ROADMAP.md:283:| HDFC parser is brittle across statement vintages | Synthetic fixtures +
the D.3 evaluation gate **before** generalizing; deterministic-first with a planned local-LLM
fallback for the long tail. |
docs\ROADMAP.md:298:[Omitted long matching line]
docs\schema.md:44:| status | TEXT | NOT NULL default `parsed`; parsers set `reconciled` /
`unreconciled` |
docs\TESTING.md:51:The corpus self-stratifies into capability tiers. Tests are gated by tier so
CI stays green and honest about what's proven.
docs\TESTING.md:156:- Coverage reported but thresholds start modest and ratchet up; we don't
gate on a vanity number before the code exists.
docs\TESTING.md:165:- ****Cloud-LLM category gate.**** Calling the cloud-LLM client with a category
in {bank, cc, broker, mf, insurance} raises/refuses; only {promo, ?explicitly-approved} are
allowed. A test asserts the refusal.

14. user (2026-06-10T16:29:58.495Z)

<system-reminder>This memory is 10 days old. Memories are point-in-time observations, not live
state ? claims about code behavior or file:line citations may be outdated. Verify against
current code before asserting as fact.</system-reminder>

```

1      ---
2      name: parser-validation-pattern
3      description: The standard structure + validation every muneem bank-statement parser must
follow
4      metadata:
5          node_type: memory
6          type: project
7          originSessionId: a9fd4426-f234-4e4a-bfb4-1f2c8255c661
8      ---
9
10     Established across HDFC, Axis, ICICI (PRs #14?#18). Every parser in `muneem/parsers/`
follows this:
11
12     1. **Split parse from persist.** A pure `_rows_from_` / `parse_` returns dataclass
rows ? unit-testable on synthetic table/word lists with **no PDF and no DB**.
`StatementParser.parse_and_insert` is a thin wrapper that inserts + validates. (Do NOT mock
pdfplumber to test parsing ? test the pure row-builder on synthetic input.)
13
14     2. **Reconcile.** Use the shared
`muneem.parsers.validation.reconcile_running_balance(entries)` where entries =
`(debit/withdrawal, credit/deposit, balance)` and None?0. A clean walk (`balance[i] ==
balance[i-1] - debit + credit`) proves no row was missed/misread **without inspecting any
value** ? it's what catches silent `extract_tables()` row-drops. Use `validation.parse_amount`
for money (keeps `0.00` = 0.0, None only for an absent cell); do **not** use

```

`base.safe\_amount`, which conflates 0 and absent and corrupts the math.

15

16 3. ****Bookend when available.**** Cross-check parsed Dr/Cr counts + opening/closing balance against the bank's own end-of-statement SUMMARY (HDFC has one: `parse\_summary` + `validate\_against\_summary`). Catches a missing first/last row that reconciliation alone cannot.

17

18 4. ****Record quality.**** `parse\_and\_insert` sets `documents.status` = `reconciled` / `unreconciled`. Credit-card statements have no running balance ? can't reconcile (count only).

19

20 5. ****Validate on real data, redacted.**** `@pytest.mark.local\_statements` tests parse real (encrypted) statements and assert reconcile/bookend; surface ****counts/booleans only****, password ****in-memory only**** (env like `MUNEEM\_TEST\_\*\_PW` or a gitignored `password\_rules.yaml`). Skip cleanly when no password.

21

22 ****Registry gotcha (fixed in #13):**** parsers self-register via `@register\_parser` on import; `parsers/\_\_init\_\_\_.autoload\_parsers()` imports every submodule so the registry is populated in the **production** path (`from muneem.parsers import get\_parser`), not just under tests. New parser files auto-register ? no edit needed.

23

24 ****HDFC is the reference**** (coordinate clustering + full reconcile + SUMMARY bookend; verified to match the bank's Dr/Cr counts and opening/closing exactly on every real statement). Axis/ICICI use `extract\_tables`; if reconciliation breaks on a real statement, that's the signal to move them to coordinate clustering like HDFC. See [\[\[hdfc-parser-tech-decision\]\]](#).

25

26 ****Axis savings ? recovered (was "broken"); now 6/7 on real data.**** Rewritten onto the concept core (#25): coordinate clustering ? generic engine mapping ? reconciliation oracle, oracle-gated (the old fixed-column `extract\_tables` mapping is gone). PR #32 added `savings.\_reconstruct\_by\_balance\_delta` ? a ****bank-agnostic**** fallback that derives debit/credit from the ****sign of the running-balance change**** and cross-checks `amount == |?balance|` (needs the printed opening; a money column = money cells outnumber date cells, so a sparse deposit column counts and an interleaved value-date column doesn't; balance = rightmost money column). It recovered ****June 2025****. Real-data: ****HDFC 6/6, AU 4/4, Axis 6/7****. Remaining miss: ****May 2025**** ? the clusterer DROPS one transaction line (balance jump matches no amount), so the oracle refuses the incomplete ledger (stores nothing); follow-up is smarter line recovery in the clusterer. HDFC/Axis/AU passwords are per-session in-memory only.

27

15. assistant (2026-06-10T16:30:07.055Z)

The project's own docs claim a "stores nothing" invariant. Let me read the relevant doc sections to see if HDFC is documented as an exception.

16. user (2026-06-10T16:30:08.470Z)

15 1. ****Cached candidates**** ? passwords from earlier successful unlocks, stored in your ****OS keychain****.

16 2. ****Configured candidates**** ? entries in `password\_rules.yaml` (see below).

17 3. ****Interactive prompt**** ? a hidden prompt (`getpass`). A password entered here that works is

18 ****cached in the keychain****, so the next ingest of any statement with the same password is

19 friction-free.

20

21 On success you'll see, e.g.:

22

23 ```

24 ingested statement.pdf (doc 1); parsed 37 transaction(s) via HDFCParser [reconciled]

25 ```

26

27 `[reconciled]` means the parse is trusted; `[unreconciled]` means it couldn't be verified, so

28 ****nothing was stored**** (muneem never persists mislabeled rows). Read parsed rows back with

29 `muneem txns list hdfc` (or `axis` / `au`).

30

```

31     ## Enable unattended unlocking (recommended)
32
33     For batch/backfill or `--no-prompt` runs, put your statement passwords in a
34     gitignored rules
35     file so you're never prompted.
36
37     1. Find your config directory:
38
39     ```bash
40     muneem info          # ? "config dir : /home/you/.config/muneem" (varies by OS)
41     ```
42
43     2. Create `/password_rules.yaml`. Never commit it ? ./.gitignore already
44     blocks
45     `password_rules.*`, and it lives in your OS config dir (outside the repo) anyway.
46
47     ```yaml
48     # <config-dir>/password_rules.yaml  ?  NEVER commit this file.
49     # Quote every password (keeps leading zeros; avoids YAML parsing "0106" as a number).
50
51     # Per-issuer passwords:
52     hdfc: "<your-hdfc-pdf-password>"
53     axis: "<your-axis-pdf-password>"
54     aubank: "<your-au-pdf-password>"
55
56     # ?or a bank-agnostic list of passwords to try (handy when the issuer label varies):
57     candidates:
58       - "<password-a>"
59       - "<password-b>"
60     ```
61
62     muneem tries all of these (it has to pick the password before it can read the PDF
63     to know
64     the issuer), so either form works. A legacy ./password_rules.yaml in the current
65     directory is
66     still read but warns ? prefer the config dir.
67
68     3. Ingest without prompts:
69
70     ```bash
71     muneem ingest statement.pdf --doc-type bank_statement --no-prompt
72     ```
73
74     --no-prompt uses only cached + configured candidates and never opens an
75     interactive prompt
76     (so it's safe in cron/CI). Without it, an unresolved password falls back to the
77     hidden prompt.
78
79     ## Security
80
81     - Nothing sensitive is committed or logged. password_rules.* is gitignored;
82     resolved passwords
83     live only in memory and the OS keychain (Windows Credential Manager / macOS
84     Keychain / Linux
85     Secret Service), via muneem.secrets.
86
87     - No decrypted copy is written to disk ? the password is handed straight to the text
88     extractor
89     and the parser in memory.
90
91     - Transaction parses are reconciliation-gated: an unverifiable parse stores nothing
92     rather than
93     risk mislabeled rows.
94
95     ## Troubleshooting
96
97     | Message | Meaning / fix |

```



```

86      |---|---|
87      | `Could not unlock <file>: no working password` | None of the cached/configured
candidates worked. Run without `--no-prompt` to type it, or add it to `password_rules.yaml`. |
88      | `No parser recognizes <file> as a supported bank statement` | The issuer isn't
supported yet (today: HDFC, Axis, AU Bank), or the text didn't match. |
89      | `? [unreconciled]` (0 transactions) | The parser couldn't verify the parse (e.g. an
Axis vintage `extract_tables` mangles); nothing was stored. See [ `parser-
architecture.md`](./parser-architecture.md). |
90
91      ## Headless Linux note
92
93      Keychain storage needs a backend. Use a desktop keyring (GNOME Keyring / KWallet), or
94      `pip install keyrings.alt` for a file-based backend. Until then, `password_rules.yaml` +
`--no-prompt`
95      works without any keychain (you just won't get prompt-caching).
96

```

17. user (2026-06-10T16:30:09.134Z)

```

95      ## 6. Bank knowledge as *data* (profiles), not code
96
97      A bank profile is a small **declarative** object (likely YAML/dict): concept?label
aliases, accepted
98      date formats, anchor strings to locate the table, account-number patterns, known quirks
(multi-account
99      sections, "Other Information" column), and the password rule. **Generic detectors run
first; the
100     profile supplies priors/hints** to disambiguate ? it is not bespoke parsing logic.
Absorbing a small
101     format change = tweak a profile (often nothing ? the generic path copes). A brand-new
issuer = add a
102     profile, and only drop to custom code if the generic ladder genuinely can't cope.
103
104     ## 7. Failsafes ? the "traps" (loud drift, never silent)
105
106     The Axis lesson: a broken parse must *announce itself*. Built-in tripwires, each ? flag
+ provenance,
107     never store-and-pretend:
108
109     - header label matches no lexicon entry ? "unknown field, mapped by structure ? review"
110     - reconciliation breaks, or parsed counts ? printed summary ? **quarantine** (store
nothing or store
111     flagged; the oracle-gated parsers already refuse to persist unreconciled rows)
112     - a profile's expected anchor is missing ? "format may have changed for <bank>"
113     - _(L3)_ VLM output won't reconcile, or disagrees with a deterministic parse ? reject
the VLM
114     - **provenance + confidence on every parsed statement:** which layer/mapping produced
it, did it
115     reconcile / bookend. Trust is *recorded*, not assumed.
116
117     _Research (done):_ VLMs' top failure modes on financial numbers are **digit
transposition, decimal/
118     sign errors, dropped/duplicated rows, and column misalignment** ? which almost always
**break the
119     reconcile identity**, so the verifier rejects them. L3 traps therefore: gate every VLM
parse on
120     reconciliation; **self-consistency** re-sample (keep the pass that reconciles); use the
per-row walk
121     to localize a failing row for re-query or human review; cloud stays off unless
explicitly approved.
122
123     ## 8. Two tiers of "semantic"
124
125     - **Tier 1 ? field semantics** (this doc): what each column *means*. The resilience win.
Build first.

```

```

126     - **Tier 2 ? transaction semantics** (later): classify each row (UPI / NEFT / ATM /
charge / interest
127     / sweep), extract merchant/counterparty, assign category. Same schema, layered on top;
ties to the
128     parked merchant-canonicalization decision (DESIGN § 8.5).
129
130     ## 9. Data-model evolution
131
132     Today: per-bank raw tables (`hdfc_transactions`, `axis_transactions`, ?) ? fine for
fidelity, but the
133     concept schema points at a **unified `transactions` table** keyed on the canonical
concepts, plus
134     `account`/`document` context and **`provenance` + `confidence`** columns. Migration is
additive (the
135     per-bank tables can become views or be backfilled); sequence it after the core lands so
we don't churn
136     storage twice.
137
138     ## 10. Implementation plan (incremental, merge-as-we-go)
139
140     1. **? Core extraction ? landed.** `validation.py` is now the verifier: the per-row
walk, **total
141     reconcile** (`opening + ?credit ? ?debit == closing`), the concept-level **bookend**
(`verify`),
142     **candidate selection** (`select_reconciling`), and a recorded **confidence** verdict
143     (`Confidence`). The concept schema lives in `concepts.py` (`ConceptRow`,
`StatementConcepts`).
144     **HDFC is refactored onto it as the reference implementation** ? it maps its rows
onto the
145     concepts and verifies through the shared verifier, with **no behavior change** (every
existing
146     test passes; the relocated bookend logic is identical).
147     2. **? Concept-mapping engine (L0/L1/L2) ? landed.** `engine.py` maps a cell-matrix onto
concepts
148     via header lexicon (L0) ? profile column-order ? content search (L2), keeping the
assignment the
149     reconciliation oracle endorses (the **self-identifying balance column**). It is
**family-agnostic**
150     ? parameterized by a `FamilySpec` (concept names + typed-row builder) and an injected
oracle, so
151     an instrument family (equity/MF/ETF/bond) reuses the same control flow with its own
concepts +
152     oracle. Issuer knowledge is `profiles.py` ? **data, not code**. The engine ships
validated on
153     synthetic fixtures; HDFC consumes its profile (account pattern) while keeping its
bespoke
154     coordinate-clustering extraction (the documented `extract_tables` exception). The
first real
155     issuers driven through the engine are **Axis + AU Bank** (step 3).
156     3. **? Axis + AU Bank rewrite onto the core ? landed.** Both run the shared engine-
driven savings
157     path (`savings.py`): per-table + coordinate-clustered candidates ? engine mapping ?
reconciliation
158     oracle (per-row walk, else total reconcile), oracle-gated. Real-data precision
(local): **AU 4/4**
159     reconcile + persist; **Axis 2/4** (Aug'24, Oct'25) ? May/Jun'25 are vintages
`extract_tables`
160     mangles, so they store nothing (the safety invariant holds). HDFC stays bespoke
clustering.
161     Recovering the remaining Axis vintages (better clustering) is follow-up work.
162     4. **L3 local-vision fallback** (research done; build deferred): **DeepSeek-OCR-3B
(MIT)**, fired
163     **only** when the deterministic ladder fails to reconcile (e.g. the Axis May/Jun
vintages); output
164     accepted only if it reconciles, self-consistency re-sample on a miss, then human

```

review. Dev RTX
165 2070S ? 4-bit + SDPA; Qwen2.5-VL-7B (Apache) is the heavier / fine-tune option on a
bigger GPU.
166 See §4, §7, and the `l3-vision-and-eval` memory.
167 5. ****Unify storage**** (§9) + begin ****Tier-2 transaction semantics****.
168
169 Each step is a green, reversible PR; the verifier guarantees we never regress into
storing garbage.

18. user (2026-06-10T16:30:53.393Z)

6:pins the resulting shape, so accidental drift and deliberate changes both surface in review.
16:Known limitation (single-user, reviewed migrations): a crash **between** applying a migration's
DDL
60: document_id INTEGER NOT NULL UNIQUE REFERENCES documents(id) ON DELETE CASCADE,

19. user (2026-06-10T16:30:54.347Z)

No matches found

20. user (2026-06-10T16:31:30.226Z)

Structured output provided successfully

21. assistant (2026-06-10T16:31:36.168Z)

Verification complete. The finding is real: HDFC persists unreconciled rows (hdfc.py:435-454)
while Axis/AU gate persistence, the read path never filters by status (db.py:437-440,
cli.py:328-361), no test or doc mitigates it ? the docs actually promise the opposite behavior ?
and the closing_balance=0.0 sub-claim was the only refuted detail (unreachable dead code).
Severity high is justified.